

CryptoChain Analyzer Dashboard — Final Report

Cryptography 2025/26 · Universidad Alfonso X el Sabio · Prof. Jorge Calvo

Author: Álvaro Santamaría Antón · GitHub: AlvaroSantamariaAnton · May 2026

CryptoChain Analyzer Dashboard is an interactive Streamlit application that connects to public Bitcoin APIs (Blockstream and mempool.space) and exposes the cryptographic mechanics of the network in real time. It implements all four required modules of the course brief — a Proof-of-Work monitor (M1), a block-header analyzer with manual SHA-256 verification (M2), a difficulty-history viewer (M3), and an AI-based anomaly detector (M4) — together with one optional module (M6) that estimates the cost and probability of a 51% attack. The dashboard auto-refreshes every 60 s and gracefully handles upstream API failures.

1. Cryptographic metrics displayed and their meaning

1.1 Difficulty and the 256-bit target

Bitcoin encodes the Proof-of-Work threshold as a 4-byte compact field called bits: the first byte is an exponent and the lower three bytes are a mantissa, decoded as $\text{target} = \text{mantissa} \cdot 2^{8 \cdot (\text{exponent} - 3)}$. M1 displays the resulting 256-bit target in full as a 64-character hex string and reports the number of leading zero bits it contains. With current network parameters the target requires 78 leading zero bits (approximately 19 leading hex zeros). A valid block hash must be strictly less than this target — the more leading zero bits required, the harder the puzzle.

1.2 Manual Proof-of-Work verification with hashlib

M2 fetches the raw 80-byte serialized header from Blockstream and computes the double SHA-256 directly with Python's standard library:

```
raw = bytes.fromhex(header_hex)
digest = hashlib.sha256(hashlib.sha256(raw).digest()).digest()
block_hash = digest[::-1].hex() # Bitcoin displays hashes byte-reversed
```

The computed value is compared against the hash reported by the API, and the decoded target is checked: $\text{hash} < \text{target}$. The 6 header fields (version, prev_block, merkle_root, timestamp, bits, nonce) are also parsed individually, with all multi-byte integers read little-endian and the two 32-byte hashes reversed for display, exactly as defined in §6 of the course notes.

1.3 Inter-block times as a Poisson process

Mining can be modelled as a sequence of independent Bernoulli trials with constant success probability — a Poisson process. The waiting time between valid blocks is therefore exponentially distributed with mean $\mu \approx 600$ s. M1 plots a histogram of the deltas between the last N blocks and overlays the empirical exponential PDF (mean from observation) and the theoretical PDF (mean = 600 s). Visible deviations come from finite-sample noise, network propagation delays, and hash-rate drift between difficulty adjustments.

1.4 Difficulty re-targeting and self-correction

Every 2016 blocks (~14 days), Bitcoin adjusts the target to keep the average block time close to 600 s. M3 plots one marker per re-target over user-selectable time windows and adds a second chart showing, for each epoch, the ratio of the observed mean block time to 600 s. Bars above 1 (red) mean blocks were slow and the next adjustment will lower difficulty; bars below 1 (green) mean blocks were fast and the next adjustment will raise difficulty. This is the negative-feedback loop described in §6.1 of the course notes.

1.5 Hash rate

The expected hash rate satisfies $H = D \cdot 2^{32} / 600 \text{ s}$. M1 reports this as $\approx 970 \text{ EH/s}$ with current parameters, consistent with estimates from Cambridge CBEI. The same formula is reused in M6 to size the attacker hardware needed for a given hashrate share q of the combined network.

2. AI model — anomaly detector for inter-block times

2.1 Choice and justification

The chosen approach is an unsupervised anomaly detector for block inter-arrival times, with the theoretical exponential distribution $\text{Exp}(\lambda = 1/\mu)$ as the baseline. Three reasons motivated this choice over the alternatives (difficulty predictor, fee estimator):

- (i) Theoretical anchor. Because mining is a Poisson process, the baseline is derived from theory, not learned from data. This gives the detector a well-defined null hypothesis and allows quantitative goodness-of-fit testing.
- (ii) No labelled data required. Bitcoin blocks carry no anomaly labels; a theory-driven baseline sidesteps the need for supervised training data.
- (iii) Interpretability. The score $S(\Delta t) = \exp(-\Delta t/\mu)$ is the survival function of the baseline distribution: it equals the probability that an honest network would produce a gap at least as long as the one observed.

2.2 Data and parameter estimation

Training data consists of inter-arrival times computed from the 500 most recent blocks fetched from the Blockstream Esplora API. The mean μ is estimated as the sample mean (the MLE for the exponential family). The decision rule flags a block when $S(\Delta t) < \alpha$, which is equivalent to $\Delta t > \mu \cdot \ln(1/\alpha)$. With $\alpha = 0.01$ and the observed $\mu \approx 583 \text{ s}$, the anomaly threshold is approximately 45 minutes between consecutive blocks.

2.3 Evaluation results

Goodness of fit — Kolmogorov-Smirnov. The maximum vertical distance between the empirical CDF and the theoretical exponential CDF, with the asymptotic Kolmogorov approximation $p \approx 2 \cdot \exp(-2 \cdot n \cdot D^2)$ as p-value. On 500 recent blocks: $D = 0.0510$, $p = 0.154$. The test does not reject the exponential model — the data is consistent with the Poisson-process baseline.

Calibration. For each candidate threshold $\alpha \in \{0.001, 0.005, 0.01, 0.02, 0.05, 0.1\}$ the empirical false-positive rate is plotted against the nominal α . The observed points tracked the perfect-calibration diagonal almost exactly across the full range, confirming that the chosen threshold has the intended statistical meaning.

At $\alpha = 0.01$ the detector flagged 3 of 493 blocks (0.61%), with inter-arrival times in the 45–48 minute range. These correspond to genuinely long real-network gaps rather than model artefacts, and the flagged count is close to the $\sim 1\%$ expected by chance under perfect calibration.

2.4 Limitations

μ is estimated from the same data used for testing, so the standard KS test is mildly conservative (a Lilliefors correction would tighten the p-value). The exponential distribution also assumes a stationary hash rate; drift within a 2016-block epoch can produce short bursts of flagged blocks that reflect mining-pool variance rather than network events. Finally, the detector is one-sided (slow blocks only); anomalously fast sequences — which could indicate selfish mining — would require a separate test.

3. Optional module — security score (M6)

To satisfy criterion C5 (metric beyond course notes), the dashboard includes a 51%-attack security calculator with two complementary outputs.

Attack cost in USD/hour. Derived from the live network hash rate (~ 970 EH/s), an attacker-share parameter q , an ASIC efficiency (default 20 J/TH), and an electricity price (default \$0.05/kWh, the industry rate for industrial mining). At $q = 51\%$ the pure-electricity cost is approximately \$987 K/hour. This excludes hardware acquisition, cooling, and opportunity cost; realistic full attack cost is one to two orders of magnitude higher.

Success probability vs confirmation depth. Implements the Poisson-based formula from the Bitcoin whitepaper (Nakamoto 2008, §11). For $q = 10\%$ the probability of a double-spend after 6 confirmations is 0.000243; for $q = 30\%$ it is 0.132; for $q = 49\%$ it is 0.959. The exponential decay with depth provides the formal justification for the standard "wait for 6 confirmations" rule used by exchanges.

4. References

- [1] Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System.
<https://bitcoin.org/bitcoin.pdf> — used in M6 for the attacker-success-probability formula in Section 11.
- [2] Calvo, J. (2025/26). Course notes, Topic 7 — Cryptography. Universidad Alfonso X el Sabio — theoretical framing of Proof of Work, the 256-bit target, and the difficulty re-target rule (§6 and §6.1).
- [3] Blockstream (2024). Esplora HTTP API Reference.
<https://github.com/Blockstream/esplora/blob/master/API.md> — used for the current chain tip, block details, and raw 80-byte block headers consumed by M2.
- [4] mempool.space contributors (2024). mempool.space REST API documentation.
<https://mempool.space/docs/api/rest> — used for the per-epoch difficulty history series consumed by M3.
- [5] Massey, F. J. (1951). The Kolmogorov-Smirnov Test for Goodness of Fit. Journal of the American Statistical Association, 46(253), 68–78 — methodological reference for the M4 evaluation metric.